
LECTURE NOTES

Basic Concepts

AI512 — Introduction to Machine Learning

Simon Holm
2026-09-04



DEPARTMENT OF MATHEMATICS
AND COMPUTER SCIENCE

Contents

1	Formal definitions	3
1.1	Formal definitions	3
1.1.1	Experience E	3
1.1.2	Task T	3
1.1.3	Performance measure P	3
2	Empirical Risk Minimization (ERM)	3
3	Polynomial curve fitting	4

1 Formal definitions

1.1 Formal definitions

“A computer program is said to learn from experience E with respect to some class of tasks T and performance measure P , if its performance at tasks in T , as measured in P , improves with experience E ”. [Mitchell, 1997]

So

1.1.1 Experience E

Experience E comes from the dataset itself $S = \{(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)\}$.

The goal is to find a hypothesis that accurately represents $S := \{(x_i, y_i) \stackrel{i.i.d}{\sim} D\}$

1.1.2 Task T

The task is to predict $f(x_1) = y_i$ with $h(x_i) \approx y_i$

$$f(x) \approx h(x), \forall x \in X$$

Task can either be a **Classification** or a **Regression** dependent on whether Y is a continuous set (like $\mathbb{R}$)

1.1.3 Performance measure P

Performance is measure via a loss(risk) function $L : Y \times Y \rightarrow \mathbb{R}^+$ Where $\ell(y, y')$

where y is the true value of y and $\hat{y}$ is the prediction of y

Loss (risk) - measures how bad a single prediction $\hat{y}$ is compared to the true label y .

We define two common choices

- $\ell(y, \hat{y}) := \mathbb{I}(y \neq \hat{y})$, zero-one loss for classification
- $\ell(y, \hat{y}) := (y - \hat{y})^2$, squared error for regression

2 Empirical Risk Minimization (ERM)

Take the zero-one loss $\ell(y, y') = \mathbb{I}(y \neq y')$ as an example

Then we want to minimize $L_{D,f}(h) := \mathbb{P}_{x \sim D}[f(x) \neq h(x)]$

$$h_* = \arg \min_h L_{D,f}(h)$$

Since $f(x)$ is unknown we can find the average using actual true results

$$h_S = \arg \min_h L_S(h) = \arg \min_h (\text{abs}(\{i \in [m], h(x) - f(x)\}))/m$$

This is not well defined and has weaknesses regarding the hypothesis. We resolve to curve fitting.

3 Polynomial curve fitting

With dataset $S = \{(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)\}$ it is wise to split data into the following:

- Training set, where the learning algorithm will find a h that minimizes $L(h)$
- Test set, check how well the algorithm is doing (are we closer to approximating $f(x)$?)
This is ofc assuming i.i.d.

We can use polynomial fitting instead

$y(x) = \sum_{m=0}^M w_m x^m$ Then by **ridge regression** $w = (X^T X + \lambda I)^{-1} X^T Y$ We can predict y by $x^T w$ (example of this)

For each order (m) we can calculate the risk with the **Root Mean Squared Error (RMSE)**

$$L_S := \sqrt{\frac{1}{m} \sum_{i \in [m]} (h(x_i) - y_i)^2}$$

(example)

This should by theory then go down (risk should decrease as the algorithm learns)

Notice that at $M = 7$ the model overfits and acts poorly

This is because of very small or large weights

- at a small m , model will likely underfit
- at a high m , model will likely overfit

To avoid this we use a **regularizer** $h_S := \arg \min_h L_S(h) + \lambda w_m^2$ This mitigates over/under-fitting as polynomial degree increases